OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 11

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

QUESTION: 1

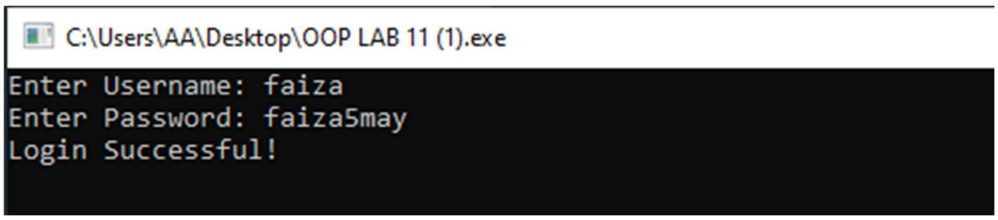
Create a C++ Program to implement login. It should accept user name and password and throw a custom exception if the password has less than 6 characters or does not contain a digit.

SOURCECODE:

```
#include <iostream>
#include <exception>
using namespace std;

// Custom Exception
class InvalidPasswordException : public exception {
public:
   const char* what() const noexcept override {
      return "Password must be at least 6 characters and contain at least one
digit.";
   }
};

// Function to validate password
void validatePassword(string password) {
   if (password.length() < 6)
      throw InvalidPasswordException();

   bool hasDigit = false;
   for (char c : password) {
      if (isdigit(c)) {
         hasDigit = true;
         break;
      }
   }
```

```
    if (!hasDigit)
        throw InvalidPasswordException();
}

int main() {
    string username, password;

    cout << "Enter Username: ";
    cin >> username;

    cout << "Enter Password: ";
    cin >> password;

    try {
        validatePassword(password);
        cout << "Login Successful!" << endl;
    }
    catch (InvalidPasswordException &e) {
        cout << "Error: " << e.what() << endl;
    }

    return 0;
}
```

OUTPUT:



QUESTION: 2

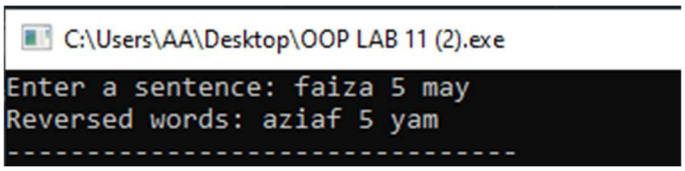
Create a class for dynamic stack using vectors. Implement the push( ), pop( ) and peek() functions. The object of stack class will be used to take any sentence as input and it should have a method reverse( ) which will reverse each word in the string.

SOURCECODE:

```
#include <iostream>
#include <vector>
#include <sstream>
using namespace std;

class Stack {
private:
   vector<char> data;

public:
   void push(char c) {
      data.push_back(c);
   }

   char pop() {
      if (data.empty()) return '\0';
      char c = data.back();
      data.pop_back();
      return c;
   }

   char peek() {
      if (data.empty()) return '\0';
      return data.back();
   }

   bool isEmpty() {
```

```
        return data.empty();
    }

    // Reverse each word
    string reverseWords(string sentence) {
        string result = "";
        for (char c : sentence) {
            if (c != ' ') {
                push(c);
            } else {
                while (!isEmpty())
                    result += pop();
                result += ' ';
            }
        }

        // Last word
        while (!isEmpty())
            result += pop();

        return result;
    }
};

int main() {
    Stack s;
    string sentence;

    cout << "Enter a sentence: ";
    getline(cin, sentence);

    cout << "Reversed words: " << s.reverseWords(sentence);

    return 0;
}
```

OUTPUT:



QUESTION: 3

You are given N integers. Store N integers in a vector and write a function Sort for sorting the N integers and print the sorted order. Now use the sorting algorithms provided in STL for sorting the vector. Measure the time taken by the two methods for sorting the vector and print the results. [Hint: You can use built-in function time( ) or anyother built-in method for measuring the processing time of sorting operation.]

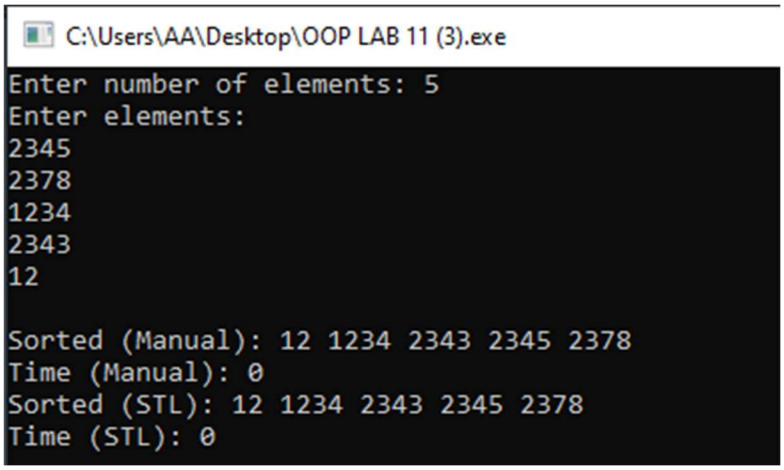
SOURCECODE:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <ctime>
using namespace std;

// Manual Bubble Sort
void bubbleSort(vector<int>& arr) {
   int n = arr.size();
   for (int i = 0; i < n-1; i++) {
      for (int j = 0; j < n-i-1; j++) {
         if (arr[j] > arr[j+1])
            swap(arr[j], arr[j+1]);
      }
   }
```

```
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> v(n);

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++)
        cin >> v[i];

    vector<int> v1 = v, v2 = v;

    // Manual sort timing
    clock_t start = clock();
    bubbleSort(v1);
    clock_t end = clock();
    double time1 = double(end - start) / CLOCKS_PER_SEC;

    // STL sort timing
    start = clock();
    sort(v2.begin(), v2.end());
    end = clock();
    double time2 = double(end - start) / CLOCKS_PER_SEC;

    cout << "\nSorted (Manual): ";
    for (int x : v1) cout << x << " ";

    cout << "\nTime (Manual): " << time1;

    cout << "\nSorted (STL): ";
    for (int x : v2) cout << x << " ";
```

```
    cout << "\nTime (STL): " << time2;

    return 0;
}
```

OUTPUT:



QUESTION: 4

You are a teacher and want to keep track of your students' grades. You decide to use the map container in C++ to store the student names as keys and their corresponding grades as values. Design and implement a program in C++ to fulfill the following requirements:

- Input: Prompt the user to enter the names and grades of multiple students. Allow the user to continue entering data until they choose to stop.
- Storage: Store the student names as keys and their grades as values in a map container.
- Retrieval: Implement a function to retrieve the grade associated with a given student's name. If the student is not found in the map, display an appropriate message.
- Update: Implement a function to update the grade of a specific student. Prompt the user to enter the new grade and update the corresponding value in the map.
- Deletion: Implement a function to remove a student and their grade from the map based on a given student's name.
- Display: Display all the student names and their corresponding grades stored in the map.

Testing: Write a program to test the functionality of your map implementation. Add multiple students with their grades, retrieve grades for specific students, update grades, delete students, and display the final list of students and their grades. As you implement the program, consider using appropriate data types, input validation, error handling, and clear user prompts. Remember to thoroughly test your implementation to ensure correctness and accuracy in storing and retrieving the student grades.

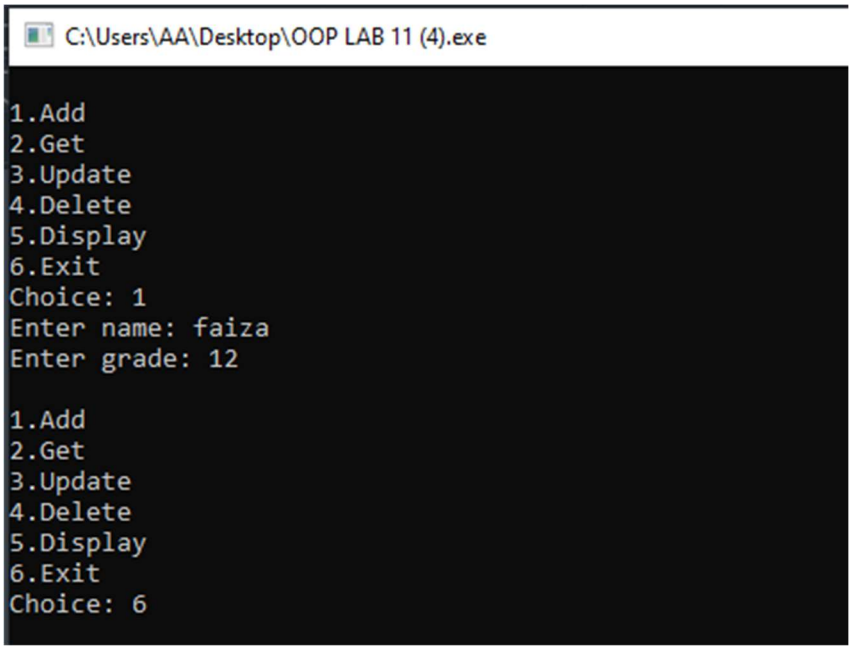
SOURCECODE:

```
#include <iostream>
#include <map>
using namespace std;

void display(map<string, float>& students) {
   for (auto s : students)
      cout << s.first << " : " << s.second << endl;
```

```
}

int main() {
   map<string, float> students;
   int choice;
   string name;
   float grade;

   do {
      cout << "\n1.Add\n2.Get\n3.Update\n4.Delete\n5.Display\n6.Exit\nChoice:
";
      cin >> choice;

      switch (choice) {
      case 1:
         cout << "Enter name: ";
         cin >> name;
         cout << "Enter grade: ";
         cin >> grade;
         students[name] = grade;
         break;

      case 2:
         cout << "Enter name: ";
         cin >> name;
         if (students.count(name))
            cout << "Grade: " << students[name];
         else
            cout << "Not found!";
         break;

      case 3:
         cout << "Enter name: ";
         cin >> name;
         if (students.count(name)) {
```

```
            cout << "New grade: ";
            cin >> grade;
            students[name] = grade;
        } else {
            cout << "Student not found!";
        }
        break;

    case 4:
        cout << "Enter name: ";
        cin >> name;
        students.erase(name);
        break;

    case 5:
        display(students);
        break;
    }

  } while (choice != 6);

  return 0;
}
```

OUTPUT:


C:\Users\AA\Desktop\OOP LAB 11 (4).exe
1.Add
2.Get
3.Update
4.Delete
5.Display
6.Exit
Choice: 1
Enter name: faiza
Enter grade: 12
1.Add
2.Get
3.Update
4.Delete
5.Display
6.Exit
Choice: 6

QUESTION: 5

You are hosting a party and want to keep track of the unique guests attending. To achieve this, you decide to use the set container in C++ to store the names of the guests. Design and implement a program in C++ to fulfill the following requirements:

- Input: Prompt the user to enter the names of multiple guests attending the party. Allow the user to continue entering names until they choose to stop.
- Storage: Store the guest names in a set container, which will automatically enforce uniqueness by discarding duplicate names.
- Display: Display all the unique guest names stored in the set in alphabetical order.
- Count: Display the total number of unique guests attending the party.

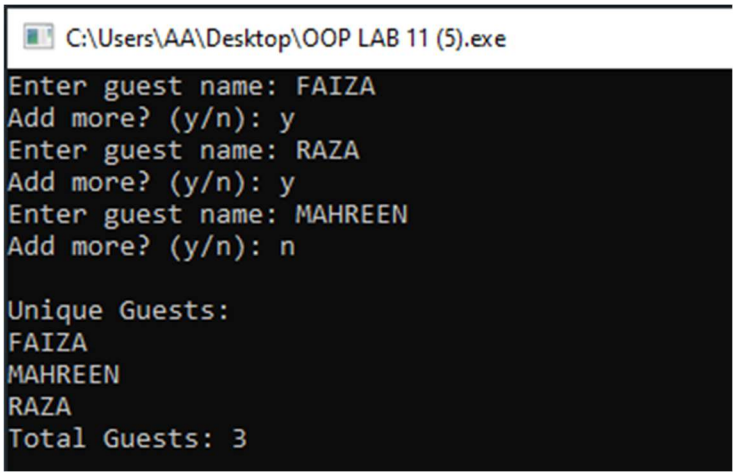
Testing: Write a program to test the functionality of your set implementation. Add multiple guest names, including duplicates, and ensure that only unique names are stored in the set. Display the final list of unique guest names and the total count of guests attending the party. As you implement the program, consider using appropriate data types, input validation, error handling, and clear user prompts.

SOURCECODE:

```
#include <iostream>
#include <set>
using namespace std;

int main() {
   set<string> guests;
   string name;
   char choice;

   do {
      cout << "Enter guest name: ";
      cin >> name;
      guests.insert(name);
```

```
        cout << "Add more? (y/n): ";
        cin >> choice;
    } while (choice == 'y' || choice == 'Y');

    cout << "\nUnique Guests:\n";
    for (auto g : guests)
        cout << g << endl;

    cout << "Total Guests: " << guests.size();

    return 0;
}
```

OUTPUT:



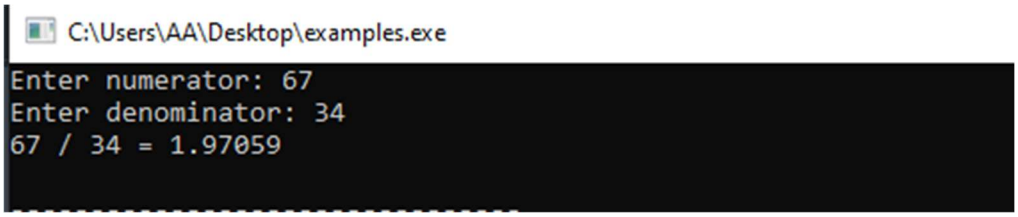
SOLVED EXAMPLE

EXAMPLE: 1

SOURCECODE:

```
#include <iostream>
using namespace std;

int main() {
   double numerator, denominator, answer;

   cout << "Enter numerator: ";
   cin >> numerator;

   cout << "Enter denominator: ";
   cin >> denominator;

   try {
      if (denominator == 0)
         throw denominator;

      answer = numerator / denominator;

      cout << numerator << " / " << denominator << " = " << answer << endl;
   }
   catch (double num_exception) {
      cout << "Error: Cannot divide by " << num_exception << endl;
   }

   return 0;
}
```

OUTPUT:

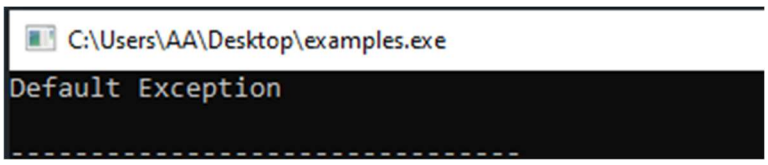


EXAMPLE: 2

SOURCECODE:

```
#include <iostream>
using namespace std;

int main() {
   try {
      throw 10;  // throwing int
   }
   catch (char c) {
      cout << "Caught char: " << c;
   }
   catch (...) {
      cout << "Default Exception" << endl;
   }

   return 0;
}
```

OUTPUT:



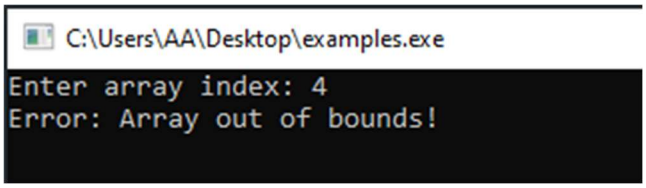
EXAMPLE: 3

SOURCECODE:

```
#include <iostream>
using namespace std;

int main() {
   double numerator, denominator, arr[4] = {0};
   int index;

   cout << "Enter array index: ";
   cin >> index;

   try {
      if (index >= 4)
         throw "Error: Array out of bounds!";

      cout << "Enter numerator: ";
      cin >> numerator;

      cout << "Enter denominator: ";
      cin >> denominator;

      if (denominator == 0)
         throw denominator;

      arr[index] = numerator / denominator;

      cout << "Result: " << arr[index] << endl;
   }

   catch (const char* msg) {
      cout << msg << endl;
   }
```

```
    catch (double num) {
        cout << "Error: Cannot divide by " << num << endl;
    }

    catch (...) {
        cout << "Unexpected error!" << endl;
    }

    return 0;
}
```

OUTPUT:



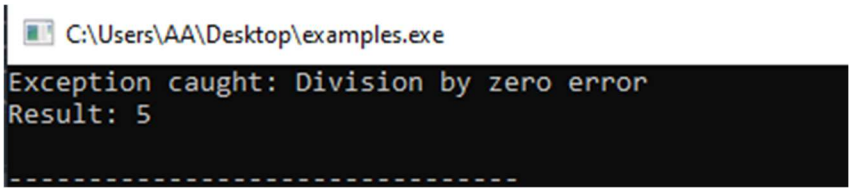
EXAMPLE: 4

SOURCECODE:

```
#include <iostream>
#include <stdexcept>
using namespace std;

void divide(int num, int den) {
    try {
        if (den == 0)
            throw runtime_error("Division by zero error");

        cout << "Result: " << (float)num / den << endl;
    }
    catch (const exception& ex) {
```

```
        cout << "Exception caught: " << ex.what() << endl;
    }
}

int main() {
    divide(10, 0);
    divide(10, 2);

    return 0;
}
```

OUTPUT:



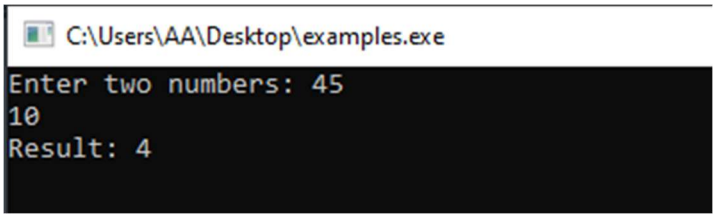
EXAMPLE: 5

SOURCECODE:

```
#include <iostream>
using namespace std;

class MyException {
public:
    const char* what() {
        return "Attempted to divide by zero!";
    }
};

int main() {
    int x, y;

    cout << "Enter two numbers: ";
```

```
    cin >> x >> y;

    try {
        if (y == 0) {
            MyException e;
            throw e;
        }

        cout << "Result: " << x / y << endl;
    }
    catch (MyException& e) {
        cout << e.what() << endl;
    }

    return 0;
}
```

OUTPUT:

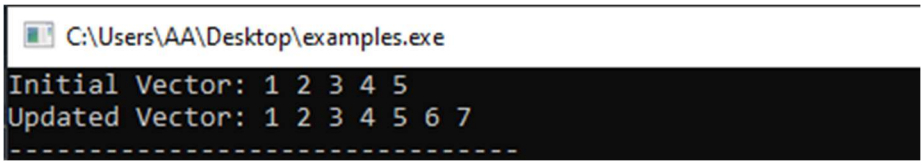


EXAMPLE: 6

SOURCECODE:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
   vector<int> num = {1, 2, 3, 4, 5};

   cout << "Initial Vector: ";
   for (int i : num)
      cout << i << " ";

   num.push_back(6);
   num.push_back(7);

   cout << "\nUpdated Vector: ";
   for (int i : num)
      cout << i << " ";

   return 0;
}
```

OUTPUT:



EXAMPLE: 7

SOURCECODE:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
   vector<int> num = {1, 2, 3, 4, 5};

   cout << "Element at index 2: " << num.at(2) << endl;

   num.at(1) = 9;

   cout << "Updated Vector: ";
   for (int i : num)
      cout << i << " ";

   return 0;
}
```

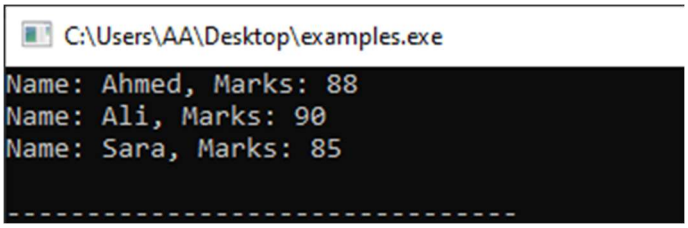
OUTPUT:

EXAMPLE: 8

SOURCECODE:

```
#include <iostream>
#include <map>
using namespace std;

int main() {
   map<string, int> m;

   m["Ali"] = 90;
   m["Sara"] = 85;
   m["Ahmed"] = 88;

   for (auto it = m.begin(); it != m.end(); it++) {
      cout << "Name: " << it->first
           << ", Marks: " << it->second << endl;
   }

   return 0;
}
```

OUTPUT:



EXAMPLE: 9

SOURCECODE:

```
#include <iostream>
#include <set>
using namespace std;

int main() {
   set<char> s;

   s.insert('G');
   s.insert('F');
   s.insert('G'); // duplicate ignored

   for (auto c : s)
      cout << c << " ";

   return 0;
}
```

OUTPUT: